

Proposta de Migração de React Native para Nativo (Kotlin/Swift)

Datacrazy

GABRIEL F. HERMENEGILDO

SUMÁRIO

1. Diagnóstico.....	3
2. Onde o nativo tem vantagem real.....	4
2.1. Performance e controle de execução.....	4
2.2. Trabalho em background.....	4
2.3. Notificações mais avançadas.....	4
2.4. Integrações profundas com o sistema.....	5
3. Nota sobre o ecossistema Expo.....	5
3.1. Exemplos encontrados.....	5
3.2. Resumo das escolhas de biblioteca.....	6
4. Impacto da decisão no produto.....	7
5. Priorização e estratégia de execução.....	7
5.1. Prioridade de recursos mobile.....	8
6. Riscos e trade offs.....	8
6.1. Possível objeção: não seria melhor fazer com calma em nativo?.....	9
7. Esforço estimado.....	10
7.1. Cenário A, migração total para nativo.....	10
7.2. Cenário B, migração parcial com módulos nativos.....	11
7.3. Cenário C, manter React Native e evoluir a base.....	11
8. Comparação direta.....	13
9. Conclusão.....	14

1. Diagnóstico

Durante a análise, naveguei por praticamente todo o aplicativo, incluindo home, leads, detalhe do lead, conversas, perfil e suporte. Também testei cenários reais, como estados de erro e situação de plano expirado. O que encontrei não aponta para uma limitação do React Native que justificaria uma reescrita.

Na prática, o aplicativo é formado principalmente por listas, formulários, navegação, operações de CRUD e comunicação com API. Não encontrei uma tela que dependesse de processamento nativo pesado, como câmera com processamento de imagem em tempo real, geolocalização de alta precisão ou cálculo local intensivo.

Também é importante separar os problemas de produto dos problemas de framework. Entre os pontos registrados na análise estão o relógio do sistema adiantado, inconsistência entre gráfico e card de atendente, erros de API sem tratamento adequado, travamento depois da troca de senha e tradução incompleta em partes do aplicativo. Esses casos estão mais relacionados a backend, tratamento de erro, estado e decisões de arquitetura de tela. Trocar React Native por Kotlin e Swift não resolveria esses problemas.

Outro fator é a stack do frontend web. O CRM utiliza JavaScript e TypeScript com Next.js, então manter React Native também permite que o time trabalhe dentro de um ecossistema de linguagem comum entre web e mobile. Isso pesa ainda mais porque o time mobile está sendo formado agora, depois de o aplicativo ter sido terceirizado.

Por fim, existe uma questão de velocidade de entrega. Neste momento, considero mais importante ter um time capaz de evoluir Android e iOS ao mesmo tempo do que buscar um ganho de performance que não apareceu como gargalo durante a análise.

A migração completa teria um custo alto e, pelo que foi observado, não atacaria os principais problemas atuais. O investimento faz mais sentido na estabilização do aplicativo,

na melhoria da arquitetura e nas novas experiências que realmente aproveitam o contexto mobile.

2. Onde o nativo tem vantagem real

Manter React Native não significa considerar o nativo inferior. Kotlin e Swift continuam tendo vantagens claras quando o aplicativo precisa de controle mais próximo da plataforma. O ponto é avaliar se essas vantagens são necessárias para o produto agora.

2.1. Performance e controle de execução

No React Native, o JavaScript roda no Hermes e conversa com a camada nativa usando a infraestrutura da Nova Arquitetura, incluindo JSI. Isso reduziu bastante a dependência da antiga ponte assíncrona, mas ainda existe uma diferença estrutural em relação a uma aplicação escrita diretamente para a plataforma.

Em um aplicativo de CRUD, listas e formulários, essa diferença normalmente não é perceptível para o usuário. Ela começa a ficar mais relevante em cenários como animações encadeadas, listas muito pesadas ou interações de alta frequência.

2.2. Trabalho em background

No Android, o WorkManager oferece uma forma confiável de agendar trabalho que precisa continuar sendo executado mesmo depois de determinadas mudanças no estado do aparelho, inclusive reinicialização. No iOS, o BGTaskScheduler é mais restritivo e oportunista, pois o próprio sistema decide quando a tarefa poderá ser executada.

Essa diferença existe independentemente de React Native. O que muda com o nativo é o nível de acesso e controle sobre essas APIs. Por isso, a arquitetura proposta para o multithreading não depende de background para garantir o envio. A fila é retomada quando o aplicativo volta ao primeiro plano ou quando a conexão retorna, enquanto o push cobre o período em que o aplicativo está fechado.

2.3. Notificações mais avançadas

Alguns recursos dependem diretamente das APIs nativas. Exemplos são canais com comportamento específico e som próprio, notificações persistentes de atendimento em curso, full screen intent no Android e Notification Service Extension no iOS.

Isso não impede o uso de React Native. Quando um desses recursos entrar no roadmap, podemos escrever ou instalar um módulo nativo específico e mantê-lo isolado da maior parte da aplicação.

2.4. Integrações profundas com o sistema

Widgets de tela inicial, Live Activities no iOS, atalhos do sistema e algumas integrações com hardware também exigem acesso nativo. São possibilidades futuras, mas nenhuma delas apareceu como necessidade atual do Datacrazy.

Em resumo, o nativo ganha em controle fino da plataforma e em acesso direto aos recursos do sistema. No cenário atual, as necessidades desse tipo são pontuais. Por isso, considero mais adequado resolver cada caso com um módulo nativo isolado do que migrar a aplicação inteira.

3. Nota sobre o ecossistema Expo

O aplicativo atual aparenta ser em React Native CLI puro, sem Expo. Por isso, considere dois cenários para as bibliotecas: manter a base atual ou adotar Expo Modules sobre essa mesma base.

Adotar Expo Modules não significa transformar o projeto em um aplicativo Expo nem trocar o fluxo de build. A ideia seria aproveitar módulos específicos do ecossistema quando eles reduzirem o custo de uma integração nativa.

Essa avaliação surgiu principalmente ao olhar para a manutenção das bibliotecas. Para algumas necessidades deste desafio, a opção tradicional do ecossistema bare está sem publicações recentes, enquanto a alternativa equivalente do ecossistema Expo continua recebendo manutenção.

3.1. Exemplos encontrados

- Notificações: o Notifee, que era uma referência para notificações locais, foi arquivado pela Invertase em abril de 2026. Entre as alternativas citadas no próprio projeto estão expo-notifications e o fork comunitário react-native-notify-kit, que já foi adaptado para a Nova Arquitetura.

- Compartilhamento do sistema: o react-native-receive-sharing-intent está sem publicação recente, enquanto o expo-share-intent continua recebendo atualizações.
- Agenda do dispositivo: o react-native-calendar-events está sem publicação recente, e o react-native-add-calendar-event recomenda o expo-calendar. Esse ponto não entra no escopo atual, mas reforça o padrão observado.
- Banco local: o react-native-quick-sqlite foi depreciado pelos próprios mantenedores, que indicam o react-native-nitro-sqlite. Na base bare, o @op-engineering/op-sqlite aparece como uma alternativa ativa. Além da presença do próprio expo-sqlite

Isso não significa que Expo torne React Native equivalente ao nativo. Por exemplo, expo-background-task e expo-task-manager encapsulam WorkManager e BGTaskScheduler, mas as limitações do iOS continuam existindo. O Expo reduz o custo de integração, mas não muda o comportamento definido pelo sistema operacional.

Também existe um limite no sentido contrário. O expo-notifications não cobre foreground service no Android nem alguns estilos avançados de notificação que uma solução como react-native-notify-kit pode preservar. Por isso, eu não adotaria uma biblioteca simplesmente por ser Expo. A escolha deve ser feita por necessidade e manutenção.

Minha recomendação é manter uma camada própria entre a aplicação e essas dependências. Dessa forma, se uma biblioteca for abandonada ou precisar ser substituída, a mudança fica concentrada em um ponto da arquitetura, sem espalhar a dependência pelo restante do código.

3.2 Resumo das escolhas de biblioteca

Necessidade	Base bare atual	Com Expo Modules
Recebimento de push	@react-native-firebase/messaging	@react-native-firebase/messaging ou expo-notifications
Exibição e agendamento de notificações	react-native-notify-kit	expo-notifications
Banco local SQLite	@op-engineering/op-sqlite	expo-sqlite ou @op-engineering/op-sqlite

Estado de rede	@react-native-community/netinfo	@react-native-community/netinfo
Compartilhamento do sistema	Implementação nativa direta	expo-share-intent

4. Impacto da decisão no produto

A escolha da tecnologia não deve ser tratada separadamente do roadmap. O objetivo do desafio é melhorar a experiência mobile, então a pergunta mais útil é onde o time terá maior retorno sobre o esforço investido.

A análise mostrou três frentes que considero prioritárias: levar o pipeline Kanban para o mobile, tornar o multiatendimento mais confiável e aproveitar recursos próprios do celular, como push, notificações locais e compartilhamento de contatos.

Nenhuma dessas frentes exige reescrever as telas atuais em Kotlin e Swift. Mantendo React Native, o time consegue trabalhar nas funcionalidades novas sem interromper o desenvolvimento para reconstruir o produto que já existe.

5. Priorização e estratégia de execução

A execução começa pela estabilização da base. A ideia é não construir funcionalidades novas sobre problemas que já foram identificados durante a análise, porque isso aumenta o custo de investigação depois e mistura causas quando algo falha.

1. Corrigir os problemas estruturais encontrados na análise, principalmente tratamento de erros, inconsistências de dados, estado de plano expirado, invalidação de sessão e revisão de i18n. Esses pontos persistiriam independentemente da tecnologia escolhida.
2. Definir a adoção ou não de Expo Modules sobre a base bare e validar as bibliotecas críticas antes de comprometer a implementação. Pelo menos uma funcionalidade proposta depende dessa decisão para ter um caminho com manutenção ativa, e decidir agora custa menos do que decidir no meio da fase.

3. Consolidar padrões de arquitetura e navegação no time que está sendo formado, principalmente a separação entre telas de edição e detalhe que hoje aparecem expandidas inline.
4. Implementar o pipeline Kanban mobile, que é a demanda de cliente citada explicitamente no desafio.
5. Reestruturar o multiatendimento com foco em offline first, sincronização, fila de envio, tempo real e push.
6. Finalizar os refinamentos, executar QA em aparelhos reais e preparar a publicação nas duas lojas.

Módulos nativos isolados continuam disponíveis como opção para necessidades futuras específicas de Android ou iOS, sem exigir que essa decisão seja tomada agora.

5.1. Prioridade de recursos mobile

A notificação push de novas mensagens é a funcionalidade que considero mais importante para o contexto mobile, porque permite responder rapidamente a um lead sem depender do navegador aberto. Ela pode ser implementada sem um módulo nativo customizado, nas duas alternativas de biblioteca consideradas, e deve ser uma das primeiras prioridades.

No futuro, caso apareça uma necessidade de acesso profundo a hardware, como câmera com processamento embarcado, sensores ou leitura de NFC, o caminho seria criar módulos nativos isolados. Hoje, nenhuma tela analisada indica que isso seja necessário.

6. Riscos e trade offs

Manter React Native reduz o custo de manutenção, mas não elimina riscos técnicos. O principal risco que vejo é a dependência de algumas bibliotecas de dispositivo e, no caso do multiatendimento, a complexidade da sincronização offline.

Esse risco pode ser controlado isolando o acesso às bibliotecas em camadas próprias e fazendo provas de conceito antes de comprometer a arquitetura em pontos de maior incerteza.

Opção	Vantagens	Desvantagens e riscos
Manter React Native	Uma base para Android e iOS, velocidade de entrega, alinhamento com Next.js e TypeScript no frontend web, e nenhum gargalo de performance relevante identificado na análise.	Não resolve bugs por si só, exige disciplina de arquitetura e acompanhamento da saúde das dependências.
Migrar parcialmente, com módulos nativos	Permite controle nativo quando realmente necessário, sem reescrever a aplicação inteira. Um módulo pode ser desenvolvido isoladamente e não precisa travar o restante do time.	Cria duas abordagens coexistindo no projeto e não existe, hoje, uma necessidade concreta que justifique esse custo em todas as áreas.
Migrar totalmente	Controle direto da plataforma, threads e ciclo de vida, com ganhos reais em cenários específicos de background ou animação complexa.	Duas bases de código, maior superfície de manutenção e estimativa de 4 a 6 meses apenas para atingir a paridade atual. Além disso, a migração não resolveria os bugs encontrados.

6.1. Possível objeção: não seria melhor fazer com calma em nativo?

Fazer com calma não é o problema. A questão é onde o tempo do time será melhor aproveitado. Manter uma base de código em vez de duas não significa reduzir testes ou revisão. Significa evitar gastar parte desse tempo sincronizando duas implementações do mesmo fluxo e usar a capacidade economizada em QA e refinamento.

Com um time recém formado, uma migração também aumenta a superfície de código que precisa ser aprendida, revisada e mantida. Os problemas encontrados na análise, como tratamento de erro e sincronização de dados, são problemas de lógica e integração. Nativo não impede esse tipo de falha.

7. Esforço estimado

As estimativas abaixo consideram um time mobile interno em formação, na ordem de quatro a cinco pessoas. Esse número não foi confirmado, então os prazos de calendário devem ser ajustados quando a composição real do time estiver definida.

As estimativas incluem tratamento de erro, estados de carregamento e vazio, testes manuais em Android e iOS e revisão. Não incluem trabalho de backend, que aparece como dependência em vários pontos da proposta.

7.1.Cenário A, migração total para nativo

Reescrever o aplicativo em Kotlin e Swift significa reconstruir cerca de 20 telas duas vezes, além de autenticação, camada de rede, design system, navegação, push e pipelines de build e publicação separados para cada plataforma.

Com quatro a cinco devs, considerando dois Android, dois iOS e uma pessoa compartilhada, a estimativa é de 4 a 6 meses até atingir a paridade com o aplicativo atual. Com dois especialistas, a referência passa para 6 a 9 meses. Com um único dev cobrindo as duas plataformas, a estimativa fica acima de 12 meses.

O ponto mais importante dessa estimativa é o retorno. Depois desse período, o produto estaria novamente entregando essencialmente o que já entrega hoje, enquanto os bugs documentados continuariam precisando ser corrigidos. Durante a migração, o app atual também precisaria ser mantido ou ficaria congelado.

Mesmo aumentando o tamanho do time, o prazo não diminui de forma proporcional. A infraestrutura de cada plataforma, como rede, autenticação, design system e navegação, possui dependências que não podem ser divididas indefinidamente.

7.2. Cenário B, migração parcial com módulos nativos

Um módulo nativo isolado pode ser desenvolvido sem bloquear o restante do time. A estimativa de referência é de 1 a 3 semanas por módulo considerando as duas plataformas. Em alguns casos, o uso de módulos já mantidos pelo ecossistema pode reduzir esse trabalho para poucos dias.

Eu trataria esse cenário como uma estratégia sob demanda. Hoje não existe uma tela do aplicativo que justifique criar esses módulos apenas por uma questão de performance.

7.3. Cenário C, manter React Native e evoluir a base

Este é o cenário recomendado. O esforço fica concentrado em estabilizar o aplicativo atual e implementar o Kanban mobile, o multiatendimento revisado e as experiências complementares.

Fase	Esforço bruto	Escopo principal
Fase 1, estabilização	4 a 6 semanas/dev	Correção dos achados de interface e comportamento, tratamento consistente de erros de API, estado de plano expirado, invalidação de sessão e revisão de i18n.
Fase 2, pipeline Kanban mobile	3 a 4 semanas/dev	Pager, colunas dinâmicas, cards, movimentação por bottom sheet com atualização otimista e rollback, reordenação dentro da coluna, estados vazios e testes de conflito de gesto.
Fase 3, multiatendimento offline first	8 a 10 semanas/dev	Modelagem do banco local, camada de repositório, emissão reativa, outbox transacional, sincronização incremental por sequência,

		realtime, push e idempotência de envio.
Fase 4, refinamento e recursos de dispositivo	4 a 6 semanas/dev	Indicadores compactos, valor monetário consistente, separação das seções inline do detalhe do lead, visibilidade de plano, lembrete local, ações de ligação e WhatsApp e criação de lead por compartilhamento.
Fase 5, QA e publicação	Cerca de 2 semanas	Testes em aparelhos reais, ajustes de performance, build de produção e submissão nas duas lojas.

Com a fase de multiatendimento estimada em 8 a 10 semanas/dev, o esforço total fica em aproximadamente 21 a 28 semanas de um desenvolvedor. Com quatro a cinco pessoas e execução parcialmente paralela, a expectativa é de aproximadamente 10 a 12 semanas de calendário.

Essa redução não é linear. A fase de estabilização funciona como pré requisito para as demais, a arquitetura do multiatendimento concentra decisões que não se dividem perfeitamente entre várias pessoas e QA depende de testes em aparelhos reais e do processo de publicação das lojas.

A organização sugerida seria dividir a fase 1 por área de tela, permitindo encurtá-la para aproximadamente duas semanas de calendário. Depois, três frentes podem avançar em paralelo: dois devs na fase 3, um na fase 2 e um na fase 4. A etapa final de QA não comprime muito porque depende do ciclo de testes e publicação.

Como referência, com dois devs a estimativa fica na faixa de 14 a 17 semanas. Com um dev, volta para aproximadamente 21 a 28 semanas. Acima de cinco devs, o ganho tende

a diminuir por causa da coordenação e da necessidade de revisão, principalmente em um time que ainda está formando seus processos.

8. Comparação direta

Considerando o mesmo time de quatro a cinco pessoas, a migração total para nativo levaria aproximadamente 4 a 6 meses apenas para chegar à paridade com o aplicativo atual. O cenário recomendado permite chegar, em cerca de dois meses e meio, a uma base estabilizada, ao Kanban mobile e ao novo multiatendimento, mantendo ainda capacidade para o roadmap seguinte.

É essa diferença de retorno sobre o esforço, e não uma preferência por React Native, que sustenta a recomendação.

Opção	Vantagem principal	Principal desvantagem
Manter React Native	Velocidade de entrega, uma base cross platform, alinhamento com o Next.js do frontend web e ausência de gargalo de performance relevante na análise.	Não resolve bugs sozinho, exige disciplina de arquitetura e acompanhamento das dependências de dispositivo.
Migrar parcialmente	Permite controle nativo quando realmente necessário, com custo menor por módulo.	Adiciona complexidade ao manter React Native e código nativo coexistindo, sem uma necessidade concreta hoje.
Migrar totalmente	Controle direto de plataforma, threads e ciclo de vida, com ganhos reais em cenários específicos.	4 a 6 meses para paridade mesmo com o time considerado, duas bases para manter e nenhum dos bugs encontrados seria resolvido pela troca de tecnologia.

9. Conclusão

Minha recomendação é manter React Native como base do Datacrazy. A análise não mostrou um gargalo de performance ou uma necessidade de plataforma que justifique uma reescrita completa.

O melhor retorno, neste momento, está em corrigir os problemas já encontrados, melhorar a arquitetura das telas existentes e investir nas funcionalidades que realmente diferenciam o mobile: um Kanban pensado para toque, um multiatendimento mais confiável, notificações e integrações com o próprio dispositivo.

Se uma necessidade nativa mais profunda aparecer no futuro, a arquitetura deve permitir resolvê-la com um módulo isolado em Kotlin ou Swift. Assim, o time mantém a produtividade do React Native sem abrir mão do controle da plataforma quando esse controle realmente for necessário.